

# 관제판

---

AI와 사람이 같이 일할 때 쓰는 교통 신호등

설치부터 사용까지 — 처음 쓰는 분을 위한 설명서

v1.3 · 2026년 9월

만든 사람 오영진 · MIT 라이선스

[github.com/jinstudy2024-ai/gwanje-board](https://github.com/jinstudy2024-ai/gwanje-board)

# 차례

1. 이 도구는 무엇인가
2. 일하는 순서 — 정한다 · 나눈다 · 잡는다 · 남는다
3. 먼저 알아둘 말 여덟 개
4. 깃허브에서 파일 받기
5. 설치하기 (약 10분)
6. 처음 열었을 때 할 일
7. 화면 다섯 개
8. 토론으로 먼저 정하기
9. AI끼리 토론시키기 (자동토론)
10. 하루가 이렇게 흘러갑니다
11. AI에게 일을 맡기는 법
12. 실제로 앱을 하나 만든다면 — 폴더·자료·토론·작업이 한 줄로
13. 막힐 때
14. 내 버전 확인하고 올리기

# 1. 이 도구는 무엇인가

먼저 어떤 상황을 막으려고 만든 물건인지부터 말씀드리겠습니다.

클로드코드에게 파일 하나를 고치라고 시켜 놓습니다. 그 사이 다른 창에서 코덱스에게 또 다른 일을 시킵니다. 그런데 코덱스도 마침 같은 파일을 엽니다. 하나가 고친 내용을 다른 하나가 통째로 덮어씹니다. 사람은 한참 뒤에야 알아챈다. "어, 아까 그 수정 어디 갔지."

사람끼리도 똑같습니다. 둘이 같은 문서를 각자 고치고 각자 저장하면, 나중에 연 사람의 내용만 남습니다.

**관제판은 그걸 막는 교통 신호등입니다.** 무언가를 건드리기 전에 "내가 이거 잡는다"고 여기에 적습니다. 끝나면 "놓는다"고 적습니다. 다른 사람과 AI는 손대기 전에 여기를 봅니다. 이미 누가 잡고 있으면 건드리지 않습니다.

## 이건 자물쇠가 아닙니다

누가 무시하고 파일을 고치면 관제판은 막지 못합니다. 내용을 합쳐 주지도, 되돌려 주지도 않습니다(그건 Git이 하는 일입니다). **"잡고 시작한다"는 습관을 다 같이 지킨다**는 전제에서만 쓸모가 있습니다. 대신 그 습관 하나로 덮어쓰기 사고는 거의 사라집니다.

**작업보드 본체**를 만드는 데 드는 것은 구글 계정 하나뿐입니다. 서버도 필요 없고 돈도 들지 않습니다.

## 단, 자동토론은 공짜가 아닙니다

뒤(9장)에 나오는 **AI끼리 자동으로 토론시키는 기능**은 다릅니다. 참가시킬 AI의 명령줄 도구를 각자 설치하고 **자기 유료 구독(클로드·ChatGPT 등)으로 로그인**해야 하며, 최소 둘이 필요합니다. 개발 도구를 쓰는 분을 위한 **선택 기능**이고, 없어도 나머지는 그대로 씁니다. "돈 안 든다"는 건 작업보드·잠금·사람 토론에 해당하는 말입니다.

## 2. 일하는 순서 — 정한다 · 나눈다 · 잡는다 · 남는다

기능을 하나씩 보기 전에, 이 도구로 일이 어떻게 굴러가는지 전체 그림부터 보시는 게 빠릅니다. 화면 다섯 개는 따로 노는 기능이 아니라 **한 바퀴로 이어집니다**.

| 단계    | 쓰는 화면           | 하는 일                                                                 |
|-------|-----------------|----------------------------------------------------------------------|
| ① 정한다 | 💬 토론            | 무엇을 할지, 어떻게 할지를 <b>먼저 글로 따져서 정합니다</b> . 사람끼리 해도 되고, AI 둘셋에게 맡겨도 됩니다. |
| ② 나눈다 | 📌 작업보드<br>📄 지시문 | 정해진 것을 <b>할 일 카드</b> 로 쪼개 담당을 붙입니다. 모두가 지킬 규칙은 지시문에 적어 둡니다.          |
| ③ 잡는다 | 🚦 신호등           | 실제로 파일을 건드리기 전에 <b>잡습니다</b> . 남이 잡고 있으면 기다립니다. 끝나면 놓습니다.             |
| ④ 남는다 | 📖 작업일지          | ①~③ 사이에 일어난 모든 일이 <b>저절로 기록됩니다</b> . 따로 적을 필요가 없습니다.                 |

**가장 흔한 오해가 여기 있습니다**. 이 도구를 "잠금 거는 프로그램"으로만 보면 절반만 쓰는 것입니다. 잠금은 ③번 한 칸일 뿐입니다. 진짜 출발점은 ①번, **무엇을 할지 정하는 자리**입니다.

### 실제로 이렇게 만들어졌습니다

이 도구의 다음 버전에 무엇을 넣을지도 토론으로 정했습니다. AI 셋에게 "공개판에서 다음에 뭘 고칠까"를 3라운드 토론시켜 나온 항목들을, 카드로 옮겨 하나씩 처리했습니다. 지금 읽고 계신 이 설명서도 그때 나온 항목 중 하나입니다.

## 토론은 두 가지입니다

이 구분이 중요합니다. 많은 분이 "토론 = 어려운 기능"으로 오해하고 건너뛴니다.

| 무엇                                | 준비물                                 | 누가 쓰나                         |
|-----------------------------------|-------------------------------------|-------------------------------|
| <b>토론 탭</b><br>사람들이 글로 의견을 주고받음   | <b>없음</b> . 설치 직후 바로 됨              | <b>누구나</b> . 혼자 생각 정리용으로도 씁니다 |
| <b>자동토론</b><br>AI 둘셋이 사람 없이 서로 반박 | AI 명령줄 도구 <b>두 개 이상</b> (각자 구독 로그인) | 그 도구를 쓰는 분만                   |

둘은 **같은 화면에 글이 쌓입니다**. 자동토론을 돌리면 AI들의 발언이 토론 탭에 한 편씩 올라오고, 사람이 그 아래 답글을 달 수도 있습니다.

자동토론 도구가 없어도 아무 지장 없습니다. ①번을 사람이 하면 됩니다. 나머지 ②③④는 똑같이 둡니다.

### 3. 먼저 알아둘 말 여덟 개

설명서를 읽다 보면 나오는 말들입니다. 여기만 이해하면 나머지는 쉽습니다.

**구글 스프레드시트** — 웹에서 쓰는 엑셀이라고 보시면 됩니다. 관제판은 **이 시트 한 장이 곧 데이터 창고**입니다. 탭(아래쪽 시트 이름들)이 표 하나하나가 됩니다. 뭔가 이상하면 시트를 직접 열어 눈으로 보고 고칠 수도 있습니다.

**앱스 스크립트(Apps Script)** — 구글이 공짜로 주는, 시트에 딸린 프로그램 넣는 자리입니다. 시트 메뉴의 **확장 프로그램** → **Apps Script** 로 들어갑니다. 여기에 관제판 코드를 붙여넣으면 그 시트가 앱이 됩니다. **코드를 직접 쓰실 일은 없습니다. 복사해서 붙여넣기만 합니다.**

**배포(Deploy)** — 붙여넣은 코드를 **인터넷 주소로 내보내는 것**입니다. 이게 제일 헷갈리는 부분인데, 중요합니다. 코드를 고치고 저장(**Ctrl + S**)만 하면 화면에는 새 코드가 보이지만 **인터넷 주소는 예전 코드를 계속 내보냅니다.** "배포"를 다시 해야 바뀝니다. 저장과 배포는 다른 일입니다.

**웹앱 주소** — 배포하면 생기는 인터넷 주소입니다. <https://script.google.com/.../exec> 처럼 생겼습니다. 이 주소가 곧 "우리 앱"입니다. 즐겨찾기 해 두고 매일 여는 주소입니다.

**토큰(Token)** — 24자리 영문·숫자로 된 **열쇠**입니다. 주소만 알아서는 아무것도 못 보고, 주소 뒤에 이 열쇠를 붙여야 화면이 열립니다. **주소와 토큰을 같이 아는 사람은 그 보드를 전부 읽고 씁니다.** 카카오톡이나 메일에 둘을 같이 붙여넣지 마세요.

**사본 만들기** — 누군가 만들어 둔 시트를 통째로 내 드라이브에 복사하는 기능입니다. **시트에 딸린 코드까지 같이 복사됩니다.** 그래서 나눠주는 사람이 "사본 링크"를 주면, 받는 사람은 클릭 한 번으로 코드까지 다 받습니다.

**자원** — 관제판에서 "잡고 놓는 대상"을 부르는 말입니다. 파일 경로가 보통이지만 무엇이든 됩니다. 폴더, 구글 문서, 발주서, 심지어 사무실 프린터도 됩니다. **다들 똑같은 이름으로 부르지만 하면** 그게 자원입니다.

**잠금** — 자원을 잡아 둔 상태입니다. 잡은 사람, 시각, 언제 풀리는지가 같이 기록됩니다. 잡아 놓고 잊어버려도 **정해진 시간이 지나면 저절로 풀립니다.**

## 4. 깃허브에서 파일 받기

나눠주는 사람에게 링크를 받으셨다면 이 장은 건너뛰어도 됩니다. 링크 하나로 코드까지 다 따라오기 때문입니다. 저장소에서 직접 받아야 하는 분만 보세요.

**깃허브(GitHub)** — 프로그램 파일을 공개해 두는 창고 같은 웹사이트입니다. 회원가입도 로그인도 필요 없습니다. 주소만 열면 누구나 보고 받을 수 있습니다. 화면의 버튼 이름이 영어로 나와도 놀라지 마세요. 누를 곳은 몇 개 안 됩니다.

관제판의 창고 주소는 이곳입니다.

```
https://github.com/jinstudy2024-ai/gwanje-board
```

### 받는 방법은 두 가지

무엇을 하실 건지에 따라 갈립니다. **작업보드만 쓰실 거면 방법 1로 충분합니다.** 아무것도 내려받지 않아도 됩니다.

| 무엇을 할 건가                  | 어떻게 받나                                       |
|---------------------------|----------------------------------------------|
| 작업보드만 쓴다                  | <b>방법 1</b> — 화면에서 파일 두 개를 복사만 합니다. 내려받기 없음. |
| 자동토론까지 간다<br>또는 점검 도구를 쓴다 | <b>방법 2</b> — 창고 전체를 압축 파일로 내려받습니다.          |

### 방법 1 — 파일 두 개만 복사하기 (제일 쉬움)

설치에 실제로 필요한 것은 파일 **두 개**뿐입니다. 내려받아서 어디 저장할 필요 없이, 화면에서 복사해 바로 붙여넣으면 됩니다.

- 1 브라우저 주소창에 아래 주소를 넣습니다. 이게 첫 번째 파일입니다.

```
https://github.com/jinstudy2024-ai/gwanje-board/blob/main/app/Code.gs
```

- 2 코드가 쭉 보입니다. 그 내용 **오른쪽 위에** 작은 아이콘들이 모여 있는데, 그중 **네모 두 개가 겹친 모양(복사)**을 누릅니다. 이걸 누르면 파일 전체가 복사됩니다.
- 3 구글 시트의 앱스 스크립트로 가서 붙여넣습니다. (다음 장 2단계)
- 4 두 번째 파일도 같은 방법으로 합니다.

```
https://github.com/jinstudy2024-ai/gwanje-board/blob/main/app/Index.html
```

### 마우스로 굽어서 복사하지 마세요

코드 왼쪽에 붙은 줄 번호까지 같이 복사돼서 프로그램이 망가집니다. 반드시 **복사 아이콘**을 쓰세요. 아이콘이 안 보이면 마우스를 코드 위로 가져가 보세요. 그때 나타납니다.

### 복사가 잘 됐는지 확인하는 법

붙여넣은 뒤 맨 윗줄이 `/**` 로 시작하면 제대로 된 것입니다. `1 /**` 처럼 숫자가 앞에 붙어 있으면 줄 번호가 섞인 것이니 지우고 다시 하세요.

## 방법 2 — 참고 전체를 내려받기

자동토론 드라이버, 점검 도구, 문서까지 전부 필요할 때입니다.

- 1 참고 주소를 엽니다. `github.com/jinstudy2024-ai/gwanje-board`
- 2 파일 목록 오른쪽 위에 있는 초록색 **Code** 버튼을 누릅니다.
- 3 펼쳐진 목록 맨 아래 **Download ZIP** 을 누릅니다. 압축 파일 하나가 내려받아집니다.
- 4 내려받은 파일(보통 다운로드 폴더에 있습니다)에 오른쪽 클릭 → 압축 풀기.
- 5 풀린 폴더를 내가 찾기 쉬운 곳으로 옮깁니다. 바탕화면이나 문서 폴더면 충분합니다.

### 폴더 이름에 주의

압축을 풀면 폴더 이름이 `gwanje-board-main` 처럼 뒤에 `-main` 이 붙어 나옵니다. 그대로 두셔도 되고 `gwanje-board` 로 바꾸셔도 됩니다. 다만 **폴더 경로에 한글이 섞여 있어도 괜찮지만, 어디에 뒀는지는 꼭 기억해 두세요.** 나중에 명령을 칠 때 그 위치를 적어야 합니다.

## 폴더 안에 뭐가 들어 있나

| 이름           | 무엇인가                                                                                                   |
|--------------|--------------------------------------------------------------------------------------------------------|
| app          | <b>앱 본체.</b> 이 안의 <code>Code.gs</code> 와 <code>Index.html</code> 을 구글 시트에 붙여넣습니다. 설치에 꼭 필요한 건 이 둘뿐입니다. |
| docs         | 설명서들. 지금 읽고 계신 이 PDF도 여기 있습니다.                                                                         |
| tools        | <b>점검 도구.</b> 뭔가 안 될 때 어디가 비었는지 알려 줍니다.                                                                |
| discuss      | 자동토론 프로그램. 자동토론을 안 쓰면 손덜 일 없습니다.                                                                       |
| client       | 명령줄에서 관제판을 부르는 프로그램. 역시 선택입니다.                                                                         |
| CHANGELOG.md | 버전별로 무엇이 바뀌었는지 적힌 목록.                                                                                  |

### 나중에 새 버전이 나오면

같은 방법으로 다시 내려받으면 됩니다. 참고의 내용은 늘 최신입니다. 다만 `discuss` 폴더에 만들어 둔 **설정 파일**은 새로 받은 폴더에 없으니, 예전 폴더에서 옮겨 오거나 다시 만드셔야 합니다.

## 5. 설치하기 (약 10분)

준비물은 구글 계정 하나입니다. 설치 방법은 두 가지인데, 나눠주는 사람에게 링크를 받았으면 A, 저장소에서 직접 받으면 B입니다.

### 시작 전에 1분만 확인하세요

회사나 학교 계정을 쓰신다면, 관리자가 앱스 스크립트를 막아 두었을 수 있습니다. 그러면 아무리 정확히 따라해도 중간에 막힙니다. 아무 구글 시트나 열어 **확장 프로그램** 메뉴에 **Apps Script** 가 보이는지 먼저 보세요. 안 보이거나 눌렀을 때 거부되면, 개인 구글 계정으로 하시는 편이 빠릅니다.

### A. 링크를 받은 경우

- 1 받은 링크를 엽니다. "사본 만들기" 화면이 뜹니다.
- 2 **사본 만들기**를 누릅니다. 내 구글 드라이브에 시트가 하나 생깁니다. 앱 코드도 같이 따라옵니다.
- 3 아래 **3단계**부터 이어서 하시면 됩니다.

### B. 저장소에서 직접 설치

앞 장(3. 깃허브에서 파일 받기)에서 `Code.gs` 와 `Index.html` 을 손에 넣은 상태라고 보고 이어갑니다.

#### 1단계 — 새 시트 만들기

구글 드라이브에서 **새로 만들기** → **Google 스프레드시트**. 이름은 **관제판** 정도로 바꿔 둡니다.

#### 2단계 — 코드 붙여넣기

- 1 시트 메뉴 **확장 프로그램** → **Apps Script**. 새 탭이 열립니다.
- 2 왼쪽 파일 목록의 `Code.gs` 내용을 **전부 지우고** (`Ctrl` + `A` → `Delete`), 저장소의 `app/Code.gs` 내용을 통째로 붙여넣습니다.
- 3 파일 목록 옆 **+** → **HTML** 을 누르고 이름을 `Index` 로 합니다(확장자는 자동). 내용을 지우고 저장소의 `app/Index.html` 을 붙여넣습니다.
- 4 위쪽 저장 (`Ctrl` + `S`).

### 복사할 때 요령

깃허브에서 파일을 열면 내용 오른쪽 위에 **복사 아이콘**(네모 두 개가 겹친 모양)이 있습니다. 그걸 쓰세요. 마우스로 클릭하면 줄 번호가 섞여 들어갑니다.

## 3단계 — 한 번 실행해서 열쇠 받기

- 1 편집기 위쪽 가운데 함수 이름이 적힌 칸에서 `setupSheets` 를 고르고 ▶ **실행**을 누릅니다.
- 2 처음이라면 권한을 묻는 창이 뜹니다. 여기서 되돌아가는 분이 가장 많으니 천천히 보겠습니다.

### "확인되지 않은 앱입니다" 라는 빨간 경고가 뜰 때

수상한 앱이어서가 아닙니다. **방금 내가 내 계정에 직접 붙여넣은 코드**라서 구글이 아직 심사한 적이 없다는 뜻입니다. 이 앱은 내 시트 밖으로 아무것도 보내지 않습니다. 이렇게 넘어가세요.

**권한 검토** → 내 구글 계정 선택 → 왼쪽 아래 **고급** 클릭 → 펼쳐진 "(안전하지 않음)으로 이동" 클릭 → **허용**.

- 3 아래쪽 **실행 로그**에 `토큰: 1a2b3c...` 24자가 찍힙니다. 시트로 돌아가면 탭이 여러 개 생겨 있고, **설정** 탭에도 같은 값이 있습니다.

### 이 토큰이 열쇠입니다

주소와 이 값을 아는 사람은 이 보드를 전부 읽고 씁니다. 아무 데나 붙여넣지 마세요. 바꾸고 싶으면 **설정** 탭의 토큰 값을 고치면 그 즉시 옛 값은 막힙니다.

## 4단계 — 인터넷 주소로 내보내기(배포)

- 1 편집기 오른쪽 위 **배포** → **새 배포**.
- 2 왼쪽 톱니바퀴 ⚙ → **웹 앱** 선택.
- 3 설정을 이렇게 맞춥니다. 설명은 아무거나( `v1` ), 다음 사용자 인증 정보로 실행: 나, 액세스 권한이 있는 사용자: 모든 사용자.
- 4 배포를 누릅니다. 권한 창이 또 뜨면 허용합니다. **웹 앱 URL** 이 나오면 복사해 둡니다.

### "모든 사용자"로 열어도 괜찮은가요

괜찮습니다. 이 앱은 **토큰 없이는 아무것도 보여주지 않습니다**. 주소만 아는 사람에게는 빈 화면입니다.

## 5단계 — 열어 보기

브라우저 주소창에 **웹앱주소 뒤에** `?token=토큰` 을 붙여 넣습니다.

```
https://script.google.com/macros/s/.../exec?token=1a2b3c...
```

화면이 뜨고 위에 탭 다섯 개가 보이면 설치가 끝난 것입니다. **이 주소를 즐겨찾기 해 두세요.** 앞으로 매일 여는 주소입니다.

## 6. 처음 열었을 때 할 일

### 먼저, 내 이름 고르기

화면 맨 윗줄에 **나는** 이라는 칸이 있습니다. 처음에는 (선택) 이라고 비어 있습니다.

#### 이걸 안 고르면 아무 버튼도 안 눌립니다

잡기도 안 되고 글도 못 올립니다. 처음 쓰는 분이 가장 많이 막히는 자리입니다. 화면을 열면 **제일 먼저 자기 이름을 고르세요.**

목록에 내 이름이 없나요? 구글 시트의 **설정** 탭에서 **행위자목록** 칸에 심표로 이어서 적으면 됩니다. 코드를 다시 배포할 필요 없이, 저장하면 바로 적용됩니다. 같이 쓸 AI 이름(클로드, ChatGPT 등)도 여기에 미리 넣어 두세요.

그 옆의 **프로젝트** 칸은 일을 나눠 담는 칸입니다. 여기를 바꾸면 화면 내용이 통째로 바뀝니다. 프로젝트 이름도 같은 **설정** 탭의 **프로젝트목록** 에서 늘립니다.

#### 이름은 글자 하나까지 똑같아야 합니다

목록에 없는 이름으로 뭔가 하려 하면 앱이 `unknown_actor` 또는 `unknown_project` 라며 거부합니다. 오타로 엉뚱한 잠금이 생기는 걸 막는 장치입니다. 앞뒤 공백도 다른 이름으로 칩니다.

### 화면이 비어 있으면 — 예시를 한 번 채워 보기

설치 직후 화면이 텅 비어 있는 것은 정상입니다. 고장이 아닙니다. 다만 빈 화면만 봐서는 감이 안 오니, 예시를 한 번 넣어 보고 눈으로 확인한 뒤 지우시길 권합니다.

앱스 스크립트로 돌아가 함수 칸에서 `seedExampleData` 를 고르고 ▶ **실행**. 앱을 새로고침하면 잠금 3개, 카드 4장, 지시문 1개, 일지 5줄이 들어와 있습니다.

#### 이미 쓰던 시트에서는 눌러도 안전합니다

내용이 하나라도 있으면 이 기능은 **아무 일도 하지 않고 그냥 끝납니다**. 예시가 섞여 들어가거나 기존 내용이 덮어쓰이는 일은 없습니다.

예시를 치우려면 시트의 **잠금·보드·일지·지시문** 탭에서 첫 줄(제목 줄)만 남기고 지우면 됩니다.

## 7. 화면 다섯 개

탭은 일하는 순서대로 놓여 있습니다. 💬 토론(정한다) → 📌 작업보드(나눈다) → 🚦 신호등(잡는다) → 📄 작업일지(남는다) → 📖 지시문(규칙).

다만 처음 열 때 보이는 화면은 신호등입니다. 매일 들어와 제일 먼저 확인할 것이 "지금 누가 뭘 잡고 있나"이기 때문입니다. 한 번 다른 탭을 고르면 다음에 열 때 그 탭이 그대로 열립니다.

### 💬 토론 — 글로 주고받는 곳

실시간 채팅이 아닙니다. 주제를 하나 정하고 그 아래에 글을 쌓아 가는 게시판입니다.

주제 칸에 새 주제를 입력하거나, 주제 보기 를 눌러 기존 주제를 고릅니다. 아래 칸에 의견을 쓰고 올리기. 작성자는 맨 위 나는 에서 고른 이름으로 들어갑니다.

나중에 자동토론을 붙이면, AI들의 발언이 바로 이 화면에 한 편씩 올라옵니다.

### 📌 작업보드 — 할 일을 나눠 담는 곳

+ 카드 추가를 누르면 입력 칸이 열립니다. 제목만 필수이고 나머지는 선택입니다. 담당(누가 할지), 우선순위(높음·보통·낮음), 관련자원(이 일에서 건드릴 파일), 메모.

카드는 할일 → 하는중 → 끝 세 칸을 오갑니다. 카드에 붙은 ◀ ▶ 버튼으로 옮깁니다.

#### 관련자원을 적어 두면 편합니다

카드에 든 자원 이름을 누르면 신호등 화면으로 넘어가면서 그 자원이 잡기 칸에 자동으로 채워집니다. "이 일 시작 → 바로 잡기"가 두 번 클릭으로 끝납니다.

### 🚦 신호등 — 제일 많이 쓰는 곳

"지금 누가 무엇을 붙잡고 있는지"를 보는 곳입니다.

#### 잡는 법

위쪽 자원 잡기 줄에 건드릴 것을 적고 잡기를 누릅니다. 칸이 셋 있습니다.

| 칸     | 무엇을 적나                                                                                                                                                                            |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 자원    | 컨드릴 대상. <code>app/Code.gs</code> 같은 파일 경로가 보통입니다. 폴더·문서·물건도 됩니다. <b>다들 똑같이 부르는 것이 중요합니다.</b> 한 사람은 <code>Code.gs</code> , 다른 사람은 <code>app/Code.gs</code> 로 적으면 앱은 서로 다른 것으로 봅니다. |
| 메모    | 무엇을 할 건지 한 줄. 안 적어도 되지만, 적어 두면 남이 "언제쯤 끝나려나" 짐작할 수 있습니다.                                                                                                                          |
| 만료(분) | 이 시간이 지나면 <b>저절로 풀립니다.</b> 잡아 놓고 잊는 일이 반드시 생기기 때문에 넣었습니다. 기본은 120분입니다.                                                                                                            |

## 잡은 다음

아래 **잡힌 자원** 목록에 내 것이 뜹니다. 버튼이 둘 있습니다.

**연장** — 아직 하는 중인데 만료가 다가올 때 누릅니다.

**놓기** — 끝났을 때 누릅니다. **이걸 눌러야 다른 사람이 잡을 수 있습니다.**

남이 잡은 것에는 **강제 놓기** 로 보이며, 이걸 **나** 만 누를 수 있습니다. AI가 잡아 놓고 세션이 죽었을 때 사람이 풀어 주라고 만든 비상구입니다.

## 차례 넘기기

아래쪽에 있는데 **이건 안내용일 뿐입니다.** 실제로 충돌을 막는 것은 잠금이고, 차례는 "지금 누구 순서인지" 서로 알자는 표시입니다.

## 작업일지 — 저절로 쌓이는 기록

여기는 **적는 곳이 아니라 보는 곳**입니다. 잡기, 놓기, 만료, 카드 추가, 카드 이동, 차례 넘기기 — 앱에서 일어나는 모든 일이 자동으로 한 줄씩 남습니다. 누가 언제 무엇을 했는지 따로 적을 필요가 없습니다.

위쪽 칸으로 메모를 직접 남길 수도 있습니다. "오늘 로트 등록 부분 끝냄" 같은 한 줄 보고용입니다. **행위자** 로 특정 사람 것만 골라 보거나, **전체 프로젝트** 를 체크해 한꺼번에 볼 수 있습니다.

## 지시문 — 참가자 전원이 지킬 규칙

+ 새 **지시문** 으로 규칙 문서를 만듭니다. "파일 고치기 전에 반드시 잡기", "끝나면 놓기와 카드 이동까지" 같은 팀 규칙을 적어 둡니다. 같은 제목으로 다시 저장하면 **버전이 올라가고 이전 내용도 남습니다.**

그리고 이 화면에  **에이전트 프롬프트 복사** 버튼이 있습니다. 다음 장에서 따로 설명하겠습니다. **이 앱에서 제일 쓸모 있는 버튼인데 제일 눈에 안 띄는 자리에 있습니다.**

## 8. 토론으로 먼저 정하기

준비물이 없습니다. 설치만 끝났으면 바로 쓸 수 있습니다.

### 왜 먼저 정하고 시작하나

AI에게 일을 시킬 때 가장 흔한 실패는 "무엇을 만들지가 흐릿한 채로 시작하는 것"입니다. 시키는 쪽도 정리가 안 된 상태로 말하니, AI는 그럴듯한 걸 만들어 오고, 받아 보고 나서야 "이게 아닌데" 합니다. 그리고 다시 시킵니다. 그 왕복이 시간과 비용의 대부분입니다.

그래서 **키보드를 잡기 전에 글로 한 번 따져 보는 자리**가 필요합니다. 토론 탭이 그 자리입니다. 혼자 쓰셔도 됩니다. 머릿속에 있는 걸 글로 꺼내 놓는 것만으로도 절반은 정리됩니다.

### 어떻게 쓰나

- 1  **토론 탭**으로 갑니다. 맨 위 **나는** 에서 내 이름이 골라져 있는지 확인합니다.
- 2 주제 칸에 **정하고 싶은 것**을 적습니다. 예: **재고앱 v2 - 무엇부터 만들까**
- 3 아래 칸에 **첫 글**을 씁니다. 그리고 **올리기**.
- 4 생각이 더 나면 같은 주제에 이어서 올립니다. 같이 쓰는 사람이 있으면 그 사람도 올립니다.

#### 첫 글에 이 네 가지를 적으면 좋습니다

**무엇을 정해야 하는지 · 지금까지 나온 안 · 걸리는 점 서너 개 · 무엇을 기준으로 고를지.**

"좋은 아이디어 있나요" 같은 첫 글은 대화가 걸뎁니다. 사람이든 AI든 마찬가지입니다.

#### 주제 이름 — 좋은 예와 나쁜 예

주제는 "무엇을 정할 건지"가 드러나는 **한 줄**로 짓습니다.

좋은: **재고앱 v2 - 어느 화면부터 · 발주 알림 방식 정하기 · 이 계산식 이대로 갈까**

나쁜: **재고앱 · 회의 · 아이디어 - 너무 넓어 AI들이 헤맵니다.**

"재고앱"은 주제가 아니라 **분야**입니다. "재고앱에서 뭘 먼저 만들까"가 주제입니다.

### 정해진 것을 일로 옮기기

토론이 끝나면 **결론**을 **토론 탭**에 다시 적지 마세요. 그 글을 다시 읽는 사람은 없습니다. 대신 두 군데로 옮깁니다.

**할 일은 작업보드 카드로.** 나온 항목마다 카드를 하나씩 만들고 담당과 우선순위를 붙입니다. 이제 "정해진 것"이 "누가 언제 할 것"이 됩니다.

**지켜야 할 규칙은 지시문으로.** "이 프로젝트에서는 파일명을 이렇게 쓴다" 같이 앞으로 계속 적용될 것들입니다. 지시문은 AI에게 넘기는 프롬프트에도 함께 들어갑니다.

## 9. AI끼리 토론시키기 (자동토론)

앞 장을 AI 둘셋에게 대신 시키는 것입니다. 사람이 할 일은 주제를 정하고 구경하는 것뿐입니다.

### 이건 준비물이 있습니다

참가시킬 AI의 **명령줄 도구**가 각자 자기 구독으로 로그인돼 있어야 합니다. 클로드코드( `claude` ), 코덱스( `codex` ), 헤르메스( `hermes` ) 중 **최소 둘**. 하나도 없어도 앞 장의 토론과 나머지 기능은 그대로 씁니다.

**공짜가 아닙니다.** 인공지능 회사에 따로 내는 이용료(API 키)를 쓰는 건 아니지만, **이미 돈 내고 쓰는 유료 구독**이 있어야 하고 토론이 돌 때 그 사용량을 씁니다.

## 먼저 — 이 기능에 필요한 AI 도구 깔기

### 📋 명령어를 정확히 복사하려면 — 이 PDF 대신 웹에서 여세요

아래 검은 상자의 명령어들은 PDF에서 마우스로 긁어 복사하면 **공백·줄바꿈이 깨질 수 있습니다**. 정확히 복사하려면 같은 문서를 인터넷에서 여세요. 각 명령어 오른쪽의 **복사 버튼**을 누르면 통째로 정확히 복사됩니다.

→ [github.com/jinstudy2024-ai/gwanje-board](https://github.com/jinstudy2024-ai/gwanje-board) 의 `docs/2_자동토론_붙이기.md` (이 줄을 클릭하면 열립니다)

자동토론은 참가시킬 AI의 **명령줄 도구(CLI)**가 컴퓨터에 깔려 있고 **내 유료 구독으로 로그인**돼 있어야 합니다. 최소 둘이 필요하고, **클로드코드 + 코덱스** 조합이 가장 흔합니다. 아직 안 깔았다면 여기부터 하세요. 이미 있으면 건너뛰어도 됩니다.

**CLI** — 검은 창(터미널)에 **명령을 글자로 쳐서** 쓰는 프로그램입니다. 마우스로 클릭하는 앱과 달리 키보드로 한 줄씩 칩니다. 아래 명령들은 그대로 복사해 붙여넣으면 됩니다.

**터미널 여는 법** — 윈도우: 시작 버튼을 누르고 `powershell` 이라고 친 뒤 **Windows PowerShell** 을 엽니다. 맥: **터미널(Terminal)** 앱을 엽니다. 검은(또는 파란) 창이 뜨면 준비된 것입니다.

### ① 클로드코드 (claude)

**Claude Pro** 또는 **Max** 구독이 있어야 합니다. 무료 플랜으로는 안 됩니다.

터미널에 아래 한 줄을 붙여넣고 Enter:

```
# 윈도우 (PowerShell)
irm https://claude.ai/install.ps1 | iex

# 맥 (터미널)
curl -fsSL https://claude.ai/install.sh | bash
```

설치가 끝나면 그 터미널 창을 닫고 새 창을 연 뒤, 아래를 쳐서 로그인합니다.

```
claude
```

브라우저가 저절로 열립니다. 내 Claude 계정으로 로그인하면 끝입니다. 잘 됐는지는 이걸로 확인합니다.

```
claude --version
```

버전 숫자(예: 2.1.211)가 찍히면 된 것입니다.

## ② 코덱스 (codex)

ChatGPT Plus 이상(Plus·Pro·Business·Edu·Enterprise)이 있어야 합니다.

코덱스는 먼저 **Node.js** 라는 것이 깔려 있어야 합니다. 없으면 **nodejs.org** 에 가서 **LTS** 버전을 받아 설치하세요(다음·다음 눌러 설치하면 됩니다). 그다음 터미널에:

```
npm install -g @openai/codex
```

설치가 끝나면 터미널을 닫고 새로 연 뒤 로그인합니다.

```
codex
```

Sign in with ChatGPT 를 골라 내 ChatGPT 계정으로 로그인하세요. 확인:

```
codex --version
```

## ③ 헤르메스 (hermes) — 선택

클로드·코덱스와 달리 별도의 오픈소스 에이전트입니다. 꼭 필요하지 않습니다 — 위 둘만 있어도 토론은 됩니다. 쓰실 거라면 헤르메스 자체 안내를 따라 설치·로그인한 뒤, 터미널에서 **hermes** 가 실행되는지 확인하세요.

**"command not found" 또는 "인식할 수 없는 명령"이 나오면**

대개 터미널을 새로 안 열어서입니다. 방금 깐 프로그램은 새 터미널 창에서만 인식됩니다. 창을 닫고 다시 열어 보세요. 그래도 안 되면 그 CLI 설치가 덜 된 것이니 위 설치 명령을 다시 실행하세요.

## 다 끝났는지 한 번에 보기

이 저장소의 점검 도구를 돌리면 무엇이 준비됐고 무엇이 비었는지 ✓/✕ 로 알려줍니다.

```
# 윈도우
powershell -ExecutionPolicy Bypass -File tools\점검.ps1

# 맥 · 리눅스
bash tools/점검.sh
```

## 어떻게 돌아가나

컴퓨터에서 프로그램을 하나 실행하면, 그 프로그램이 참가자를 한 명씩 차례로 부릅니다. 각 AI에게 **"지금까지의 토론"**을 보여주고 **다음 한 마디를 받아** 토론 탭에 올립니다. 그걸 정해진 바퀴 수만큼 반복합니다.

돌아가는 동안 앱의 토론 탭을 열어 두면 **글이 하나씩 쌓이는 걸 실시간으로 봅니다**. 마음에 안 들면 그 창에서 **Ctrl+C**로 언제든지 멈춥니다.

누군가 발언 맨 앞에 **[합의]**를 붙이면 그 자리에서 끝납니다.

## 세 가지 안전장치

**성급한 합의를 막습니다**. 그냥 두면 AI들은 두세 번 만에 서로 동의하고 끝내 버립니다. 그래서 **처음 몇 바퀴는 합의를 금지**하도록 설정할 수 있습니다.

**호출 수에 상한이 있습니다**. 잘못 눌러도 정해진 횟수를 넘겨 부르지 않습니다. 시작 화면에 "예상 몇 회, 상한 몇 회"가 찍힙니다.

**시작 전에 참가자를 점검합니다**. 설치는 됐는데 로그인 안 된 AI가 흔한데, 그대로 시작하면 토론이 중간에 끊깁니다. 그래서 시작 전에 각자를 한 번씩 짧게 불러 보고, 대답 못 하면 이유를 알려 주고 뺍니다. 남은 인원으로 계속 갑니다.

## 가장 중요한 설정 하나

참가자마다 **서로 다른 관점**을 주는 것입니다. 이게 같으면 토론이 아니라 맞장구가 됩니다.

### 관점 예시

**참가자 A** — 현장운영·리스크 관점. 실제로 안 쓰게 될 위험, 예외 상황, 운영 부담을 파고든다.

**참가자 B** — 사업·우선순위 관점. 진짜 문제가 맞는지, 1차 범위와 성패 조건을 파고든다.

**참가자 C** — 데이터·자동화 관점. 데이터 구조, 계산 규칙, 자동화 가능성을 파고든다.

## 준비와 실행

설정 파일 하나를 만들고 명령 한 줄을 치면 됩니다. 자세한 절차는 저장소의 `docs/2_자동토론_붙이기.md` 에 있습니다. 그 전에 아래 점검을 돌리면 지금 무엇이 준비됐는지 한 번에 알려 줍니다.

```
# 윈도우
powershell -ExecutionPolicy Bypass -File tools\점검.ps1

# macOS · Linux · Git Bash
bash tools/점검.sh
```

### 돌리기 전에 씨앗 글을 하나 깔아 주세요

프로그램은 게시판에 쌓인 글만 읽습니다. 사람이 첫 글을 안 깔면 "좋은 지적입니다" 류의 빈 대화가 됩니다. 앞장의 네 가지(무엇을 정할지·지금까지 나온 안·쟁점·기준)를 첫 글로 올려 두고 시작하세요.

### 주제는 두 곳에 — 글자 하나까지 똑같이

같은 주제를 **앱 토론 탭**과 **설정 파일**( `자동토론.config.json` 의 주제 ) **두 곳에 똑같이** 적어야 합니다. 띄어쓰기 하나, 특수문자( — 같은 것) 하나라도 다르면 프로그램은 **다른 주제로 보고 빈 게시판에 대고 토론**합니다. 그래서 **앱에 적은 주제를 그대로 복사해 설정 파일에 붙여넣으세요**. 손으로 두 번 치지 마세요.

## 10. 하루가 이렇게 흘러갑니다

기능 설명만으로는 감이 안 오니, 실제 흐름을 통째로 적습니다.

아침에 즐겨찾기 해 둔 주소로 앱을 엽니다. 프로젝트를 고르고 **나는** 에서 내 이름을 고릅니다.

어제 토론 탭에 "다음 화면을 어떤 순서로 만들까"라는 주제를 열어 두었습니다. 밤사이 AI 셋이 돌며 올린 글이 열 편쯤 쌓여 있습니다. 읽어 보니 쟁점이 둘로 좁혀졌습니다. **여기서 오늘 할 일이 정해집니다.** 작업보드로 넘어가 그 결론을 카드 세 장으로 옮깁니다.

신호등 화면을 보니 **클로드**가 **Code.gs** 를 두 시간째리로 잡고 있고, 메모에 "로트 등록 함수 수정 중"이라고 적혀 있습니다. 그럼 그 파일은 오늘 오전엔 건드리지 않습니다. **이 확인 하나가 오늘의 사고 하나를 막은 것입니다.**

그중 내가 맡을 카드를 봅니다. 제목은 "설치가이드 갱신", 담당은 나, 관련자원은 **설치가이드.md** . 카드에 쓴 자원 이름을 누르니 신호등 화면으로 넘어가면서 그 파일이 잡기 칸에 채워집니다. **잡기**를 누릅니다. 이제 그 파일은 내 것입니다.

일을 합니다. 두 시간이 다 되어 가는데 아직 안 끝났으면 **연장**을 누릅니다.

끝나면 신호등에서 **놓기**, 작업보드로 가서 카드를 ► 로 "끝"까지 옮깁니다. 작업일지에 들어가 보면 오늘 한 일이 전부 줄로 남아 있습니다. 따로 적은 적이 없는데요.

오후에 보니 **ChatGPT** 가 잡아 놓은 파일이 있는데 만료 시각이 한참 지났는데도 "작업중"입니다. 세션이 죽은 모양입니다. **나는** 을 **나** 로 두고 **강제 놓기**를 누릅니다. 이것도 일지에 남습니다.

## 11. AI에게 일을 맡기는 법

사람이 대신 잡아 주는 것보다, AI가 스스로 잡게 만드는 쪽이 훨씬 편합니다.

지시문 화면의  에이전트 프롬프트 복사 버튼을 누르면, AI에게 줄 안내문이 통째로 복사됩니다. 앱 주소, 지금 프로젝트, 이름이 이미 채워진 상태로요.

그걸 클로드코드나 코덱스 세션 맨 처음에 붙여넣으면, 그 AI가 알아서 이 보드를 읽고, 건드릴 파일을 잠그고, 끝나면 놓고, 일지를 남깁니다. 사람이 매번 시킬 필요가 없어집니다.

### 복사하기 전에 이름부터 바꾸세요

복사되는 내용은 지금 화면의 프로젝트와 "나는" 기준입니다. 클로드코드에게 줄 거라면 나는 을 **클로드** 로 바꾼 뒤에 복사하세요. 그러지 않으면 그 AI가 사람 이름으로 잠금을 겁니다.

## 12. 실제로 앱을 하나 만든다면

지금까지 화면을 하나씩 봤습니다. 이 장은 그 조각들이 **실제 개발에서 어떻게 한 줄로 이어지는지**를 처음부터 끝까지 보여줍니다. 이걸 보면 "이 도구를 왜 쓰는지"가 확 잡힙니다.

예로 **재고 관리 앱**을 하나 만든다고 합시다.

### ① 자료를 한 폴더에 모읍니다

AI에게 앱을 만들게 하려면, AI가 **실제 자료를 보고** 일해야 합니다. 회의록·기존 코드·데이터·초안을 안 보고 시키면 엉뚱한 걸 만들어 옵니다. 그래서 먼저 **프로젝트 폴더 하나**에 관련 자료를 다 모읍니다.

```
D:\프로젝트\재고앱\  
  회의록.md  
  기존코드.gs  
  데이터샘플.csv  
  화면초안.html
```

이 폴더가 **작업의 중심**입니다. 앞으로 토론도 이 자료를 근거로 하고, 실제 개발도 이 폴더 안에서 일어납니다.

### ② 무엇을 만들지 토론으로 정합니다

바로 코딩부터 시키지 않습니다. **무엇을 먼저 만들지**를 정하는 게 먼저입니다. 앱의 💬 토론 탭에 주제를 열고, 씨앗 글에 "무엇을 정할지·후보·쟁점·기준"을 적습니다(8장).

AI끼리 토론시킬 거라면(9장), 설정 파일의 **참고파일**에 ①에서 모은 자료 경로를 넣습니다. 그러면 AI들이 **그 자료를 먼저 읽고** 토론합니다.

```
"참고파일": [  
  "D:\\프로젝트\\재고앱\\회의록.md",  
  "D:\\프로젝트\\재고앱\\기존코드.gs"  
]
```

토론이 끝나면 "발주 알림 화면부터 만들자" 같은 결론이 나옵니다.

### ③ 결론을 카드로 나눕니다

정해진 것을 📌 작업보드에 카드로 옮깁니다. "발주 알림 화면 만들기 — 담당 클로드", "데이터 연결 — 담당 코덱스" 처럼요. 이제 "무엇을 할지"가 "누가 무엇을"이 됩니다.

## ④ 그 폴더에서 AI를 돌립니다 — 여기가 핵심

이제 진짜 개발입니다. ①에서 만든 폴더에서 클로드코드를 실행합니다.

```
cd "D:\프로젝트\재고앱"  
claude
```

클로드코드는 **실행된 그 폴더를 자기 작업 공간으로 삼습니다**. 폴더 안의 파일을 읽고, 고치고, 새로 만듭니다. 앱을 만든다는 건 결국 이 폴더 안에서 AI가 파일을 주무르는 일입니다.

### 바로 여기서 사고가 납니다

클로드코드에게 `기존코드.gs` 를 고치라고 시켜 놓고, 다른 창에서 코덱스에게도 뭔가 시킵니다. 둘 다 **같은 폴더의 같은 파일**을 엽니다. 하나가 고친 걸 다른 하나가 덮어씁니다. **관제판은 정확히 이걸 막으려고 있습니다**.

## ⑤ 고치기 전에 잡고, 끝나면 놓습니다

그래서 AI에게 일을 시킬 때, 관제판에서 그 파일을 먼저 **잡습니다**(🚦 신호등). AI에게는 지시문 화면의 **프롬프트 복사**(11장)로 "고치기 전에 관제판에서 잡아라"를 넣어 두면, AI가 스스로 잡고 놓습니다.

그러면 코덱스는 그 파일을 열기 전에 관제판을 보고, 클로드가 잡고 있으면 **건드리지 않습니다**. 덮어쓰기 사고가 사라 집니다.

## ⑥ 모든 게 일지에 남습니다

누가 무엇을 언제 잡고 고치고 놓았는지, 📅 작업일지에 저절로 쌓입니다. 나중에 "이 파일 누가 언제 바꿨지"를 따로 안 물어도 됩니다.

### 한 줄로 정리하면

폴더에 자료를 모으고 → 토론으로 뭘 만들지 정하고 → 카드로 나누고 → 그 폴더에서 **AI를 돌리며** → 고치기 전에 **잡고** 끝나면 **놓고** → 모든 게 **일지**에 남습니다. 폴더는 작업이 일어나는 곳, 관제판은 그 안에서 누가 뭘 잡고 있는지 보는 신호등입니다. 이 둘이 한 세트입니다.

### 안전 — 중요한 폴더에서 그냥 돌리지 마세요

자동토론과 AI CLI는 사람 없이 돌리려고 **승인 절차를 건너뛰는 옵션**으로 실행됩니다. 그 폴더의 파일을 AI가 마음대로 건드릴 수 있다는 뜻입니다. 그래서 **지울 수 없는 중요한 원본이 든 폴더에서는 돌리지 말고**, 개발용 자료는 따로 한 폴더에 복사해 모아 두고 거기서 작업하세요.

## 13. 막힐 때

| 증상                                                           | 원인과 해결                                                                   |
|--------------------------------------------------------------|--------------------------------------------------------------------------|
| 아무 버튼도 안 눌린다                                                 | 맨 위 <b>나는 이 (선택)</b> 인지 보세요. 가장 흔한 원인입니다.                                |
| 화면에 <code>unauthorized</code> 만 나온다                          | 주소 끝에 <code>?token=...</code> 을 안 붙였거나 토큰이 틀렸습니다.                        |
| <code>unknown_actor /</code><br><code>unknown_project</code> | 시트 <b>설정</b> 탭 목록에 그 이름이 없습니다. 오타·앞뒤 공백을 확인하세요.                          |
| <code>locked</code> 라고 거부된다                                  | 남이 잡고 있습니다. <b>정상 동작입니다.</b> 기다리거나, 급하면 <b>나</b> 로 강제 놓기.                |
| 화면이 텅 비어 있다                                                  | 설치 직후엔 정상입니다. <code>seedExampleData</code> 로 예시를 한 번 넣어 보세요.             |
| 코드를 고쳤는데 화면이 그대로다                                            | <b>저장만 하고 배포를 안 한 것</b> 입니다. 배포 → 배포 관리 → 연필(편집) → 버전: <b>새 버전</b> → 배포. |
| 실행 로그에 토큰이 안 찍힌다                                             | <code>setupSheets</code> 가 아닌 다른 함수를 실행했을 수 있습니다.                        |
| 내가 잡은 걸 깜빡했다                                                 | 만료 시간이 지나면 저절로 풀립니다. 그래서 만료를 너무 길게 잡지 않는 편이 낫습니다.                        |
| 원인을 모르겠다                                                     | 아래 점검을 돌리면 어디가 비었는지 한 번에 알려 줍니다.                                         |

```
# 윈도우
powershell -ExecutionPolicy Bypass -File tools\점검.ps1

# macOS · Linux · Git Bash
bash tools/점검.sh
```

이 점검은 아무것도 바꾸지 않습니다. 준비물·설정값·앱 연결·버전·이름이 맞는지만 보고 ✓ / ~ / ✕ 로 알려 줍니다.

## 14. 내 버전 확인하고 올리기

앱 화면 **오른쪽 위**에 `v1.3` 처럼 버전이 적혀 있습니다. 지금 내 시트에 들어 있는 코드의 버전입니다. 저장소의 `CHANGELOG.md` 맨 위 번호와 다르면 옛 버전입니다.

### 새 사본을 만들 필요는 없습니다

쓰던 시트에 **코드만 갈아끼우면** 됩니다. 입력해 둔 잠금·카드·일지·지시문·토론은 전부 남고, 토큰도 앱 주소도 바뀌지 않습니다.

- 1 시트 → 확장 프로그램 → Apps Script.
- 2 `Code.gs` 를 새 내용으로 교체. `Index` 도 같은 방법으로 교체.
- 3 저장(`Ctrl + S`).
- 4 배포 → **배포 관리** → 기존 배포의 **연필(편집)** → **버전: 새 버전** → 배포.

### 4단계에서 "새 배포"를 누르면 안 됩니다

"새 배포"는 주소가 새로 생깁니다. 그러면 즐겨찾기 해 둔 옛 주소는 옛 코드를 계속 보여주고, 본인은 업데이트한 줄 압니다. 반드시 **배포 관리** → **편집** → **새 버전** 으로 가세요.

배포가 끝나고 앱을 새로고침하면 오른쪽 위 번호가 바뀌어 있어야 합니다. 그대로라면 4단계가 안 된 것입니다.

## 마지막으로, 딱 세 가지만

**하나.** 고치기 전에 잡습니다. 이 습관 하나가 이 도구의 전부입니다.

**둘.** 끝나면 놓습니다. 안 놓으면 남이 못 씁니다. 잊어도 만료로 풀리긴 하지만 그동안 남이 기다립니다.

**셋.** AI에게는 지시문 화면의 프롬프트 복사 버튼을 씁니다. 사람이 대신 눌러 주는 것보다 훨씬 편합니다.

이 문서와 프로그램은 MIT 라이선스로 공개돼 있습니다. 자유롭게 쓰고 고치고 나눠주셔도 됩니다.

[github.com/jinstudy2024-ai/gwanje-board](https://github.com/jinstudy2024-ai/gwanje-board)